- A tree is a *complete tree* if it has height `H` and every leaf node is either at level `H` or `H-1`. A complete tree is also perfectly balanced.

There are two definitions for tree *balancedness*: *height-balancedness*, which is the one used in the definitions provided, and *weight-balancedness*. They are independent characteristics of a tree, as neither of them imply the other. Both could lead to similar result, but the former is normally used, and we'll just refer to it, in the rest of this book, when we talk about balanced trees.

Binary search trees are recursive structures. Each *BST*, in fact, can be either:

- An empty tree.
- A node with a key, and a left and right subtree.

This recursive nature allows for intuitive recursive algorithms for all the basics operations on *BST*s.

Binary search trees offer a compromise between the flexibility and performance of insertions in a linked list and the efficiency of search in an ordered array. All the basic operations (`insert`, `delete`, `search`, `minimum` and `maximum`, `successor` and `predecessor`) require examining a number of nodes proportional to the height of the tree.

Therefore, the shorter we manage to keep the height of a tree, the better performance on those operations will be.

For a binary tree with `n` nodes, `log(n)` is the shortest height possible.

Let’s consider a binary tree. There can be only one root, obviously. It can have at most 2 children, so there can be at most 2 nodes with height 1. Each of them can have 2 children, so there can be at most $4=2^2$ nodes with height 2. How many nodes with height 3 can a BST have? As you probably guessed it’s $2^3=8$. Going down the tree, at each level, we increase the height by one and double the number of nodes that current level can hold.